International Cybersecurity and Digital Forensics Academy (ICDFA)

CIP-B101: Basic Computer Skills for Digital Forensics

Laboratory 2B: Advanced Linux Commands, Standard Streams and Controlled Evidence Transfer

Student: Almustapha Yusuf
Date: August 15, 2026

This laboratory exercise demonstrates standard input/output/error redirection, cryptographic hash verification, and controlled evidence file transfer using netcat between a Kali Linux virtual machine and a Windows 10 host. All transfers were verified for forensic integrity using SHA256 and MD5 hashes.

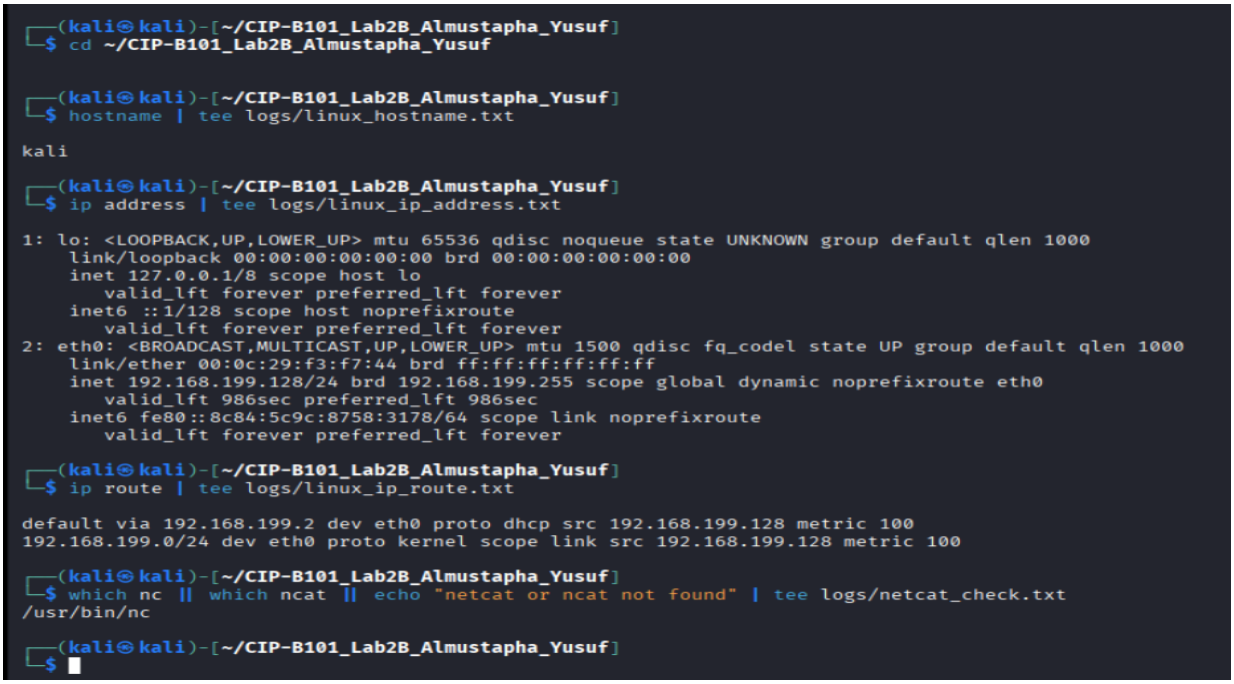
Lab Objective

The primary objective of this laboratory exercise is to develop practical competency in safe command-line evidence handling between two authorized lab machines. This lab trains students to understand how Linux treats input and output devices as files using file descriptors, how command output can be redirected into evidence logs for forensic documentation, and how files can be transferred across a controlled network without altering their content. A critical focus is placed on verifying evidence integrity before and after transfer using cryptographic hashes (SHA256 and MD5), ensuring that any transferred evidence remains forensically sound and admissible. Students also learn to use netcat/ncat as a controlled file transfer tool within an authorized lab environment, and create small test evidence images using dd for disk acquisition simulation.

Lab Environment

The laboratory environment consisted of two networked machines communicating over a VMware NAT virtual network on the 192.168.199.0/24 subnet. The Linux machine (Kali Linux) served as the primary evidence acquisition platform, while the Windows machine (Windows 10 Pro) served as the receiving endpoint. Both machines were authorized lab systems owned and operated by the student. The table below summarizes the environment details recorded before any evidence transfer operations began, following forensic best practices of documenting the acquisition environment prior to evidence handling.

Property	Kali Linux (VM)	Windows 10 (Host)
Hostname	kali	DESKTOP-OG1A0K3
Operating System	Kali Linux	Windows 10 Pro Build 19045
IP Address	192.168.199.128/24	192.168.199.1/24 (VMnet8)
Default Gateway	192.168.199.2	N/A (host adapter)
Network Adapter	eth0 (00:0c:29:f3:74:44)	VMware VMnet8
Netcat/ncat	/usr/bin/nc	C:/Program Files (x86)/Nmap/ncat.exe
Hash Tools	sha256sum, md5sum	certutil -hashfile
Connectivity	Ping: 0% loss, 1-2ms	Ping: 0% loss, 1-2ms



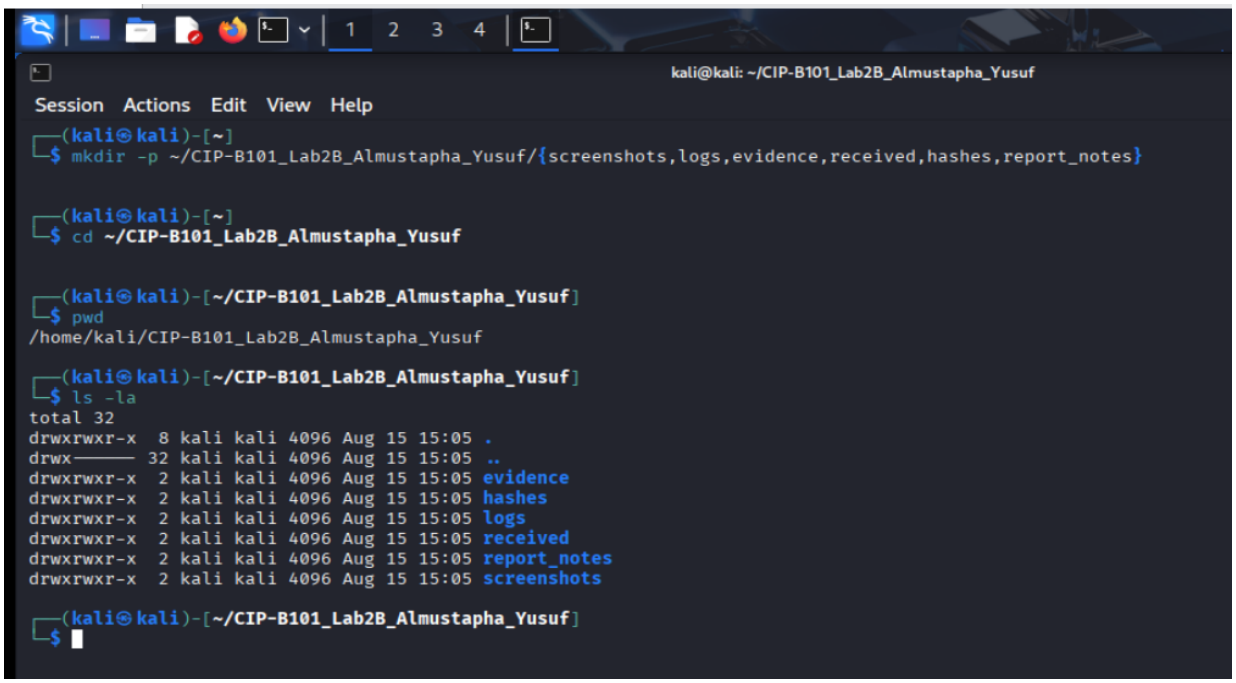
Screenshot 5: Kali Linux network details - IP address, routing, and netcat confirmation

Part A: Standard Input, Output and Error

In Linux and Unix-like systems, standard input (stdin), standard output (stdout), and standard error (stderr) are represented using file descriptors 0, 1, and 2 respectively. This abstraction is critically important in digital forensics because command output can be preserved as evidence logs, error messages can be captured separately for troubleshooting, and combined output can be redirected to create comprehensive audit trails. Understanding these file descriptors allows forensic investigators to reliably capture and document the output of every command they execute during an investigation, creating a verifiable record of their forensic actions.

A.1 Lab Folder Structure

The evidence folder structure was created with six subdirectories: screenshots, logs, evidence, received, hashes, and report_notes. This organization follows forensic best practices by segregating different categories of files to prevent cross-contamination and maintain chain of custody. The evidence directory holds source files to be transferred, the received directory holds incoming files, the hashes directory stores cryptographic hash values for integrity verification, and the logs directory preserves command output as forensic documentation.



Screenshot 1: Lab folder structure created with mkdir -p

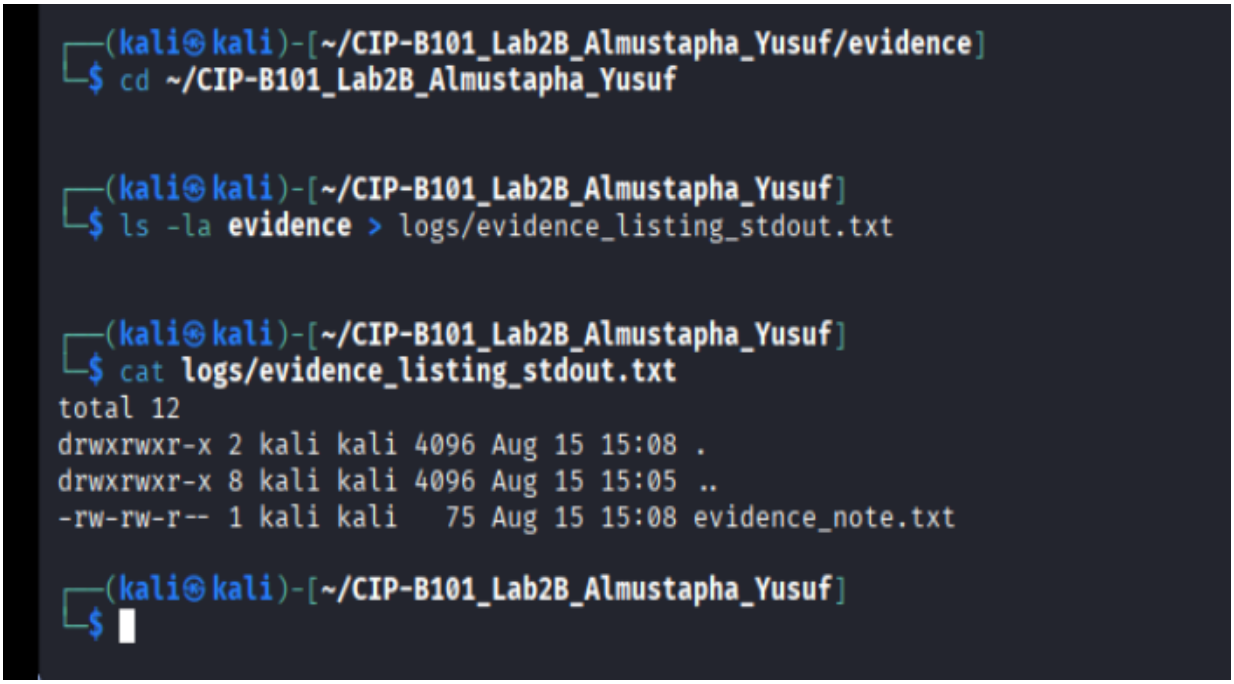
A.2 Evidence File Creation

A test evidence file was created containing the text "This is an authorized forensic evidence transfer test for Almustapha Yusuf". The file evidence_note.txt is 75 bytes in size and will serve as the primary evidence file for demonstrating controlled file transfer and hash verification in subsequent parts of this laboratory exercise.

Screenshot 2: Evidence file created and displayed

A.3 Standard Output Redirection

The > symbol redirects standard output (file descriptor 1) to a file instead of the terminal screen. In this task, the output of "ls -la evidence" was redirected to logs/evidence_listing_stdout.txt. The > operator creates a new file (or overwrites an existing one) with the command output. This is useful for evidence documentation because it creates a persistent, tamper-evident record of command output that can be included in forensic reports and referenced in legal proceedings. The content of the redirected file was verified using cat, confirming the directory listing was captured.

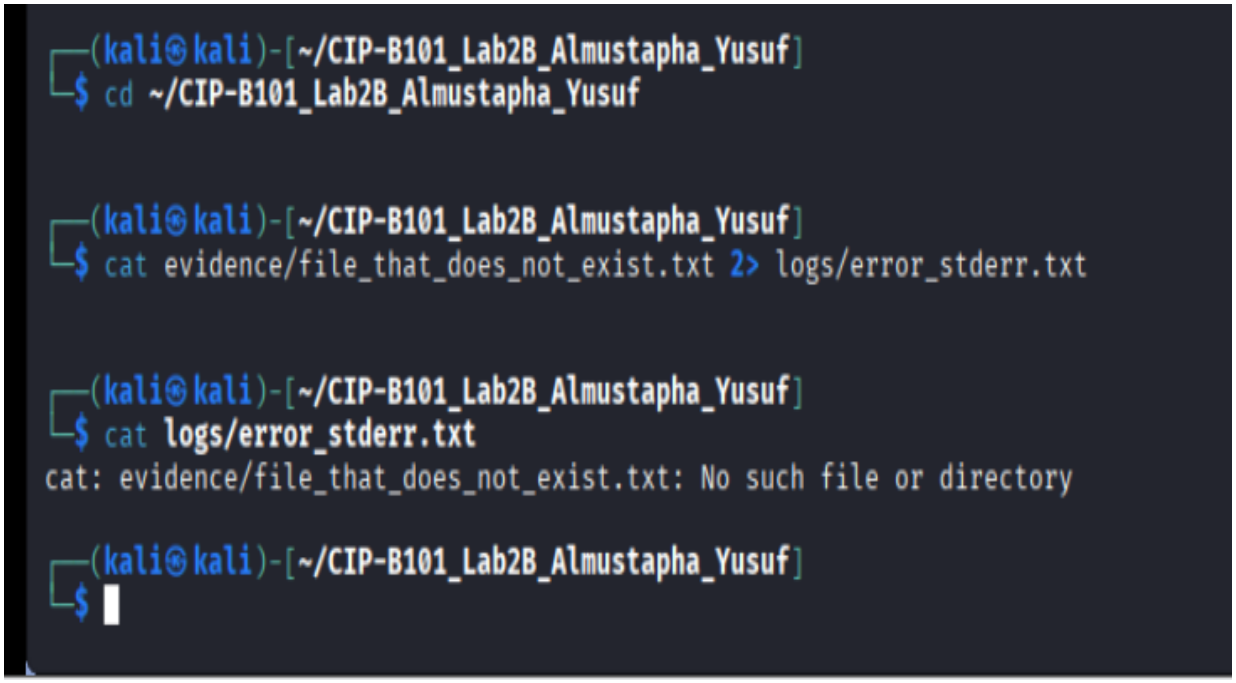


Screenshot 2.1: Standard output redirected to evidence_listing_stdout.txt

A.4 Standard Error Redirection

Standard error (file descriptor 2) carries error messages separate from normal output. By using 2>, error messages were redirected to logs/error_stderr.txt. The command "cat

evidence/file_that_does_not_exist.txt" produced an error message "No such file or directory", which was captured in the error log file. This separation of stdout and stderr is forensically valuable because it allows investigators to document both successful command outputs and error conditions in separate, organized log files, making it easier to troubleshoot and audit the forensic examination process.



Screenshot 3: Standard error redirected to error_stderr.txt

A.5 Combined stdout and stderr Redirection

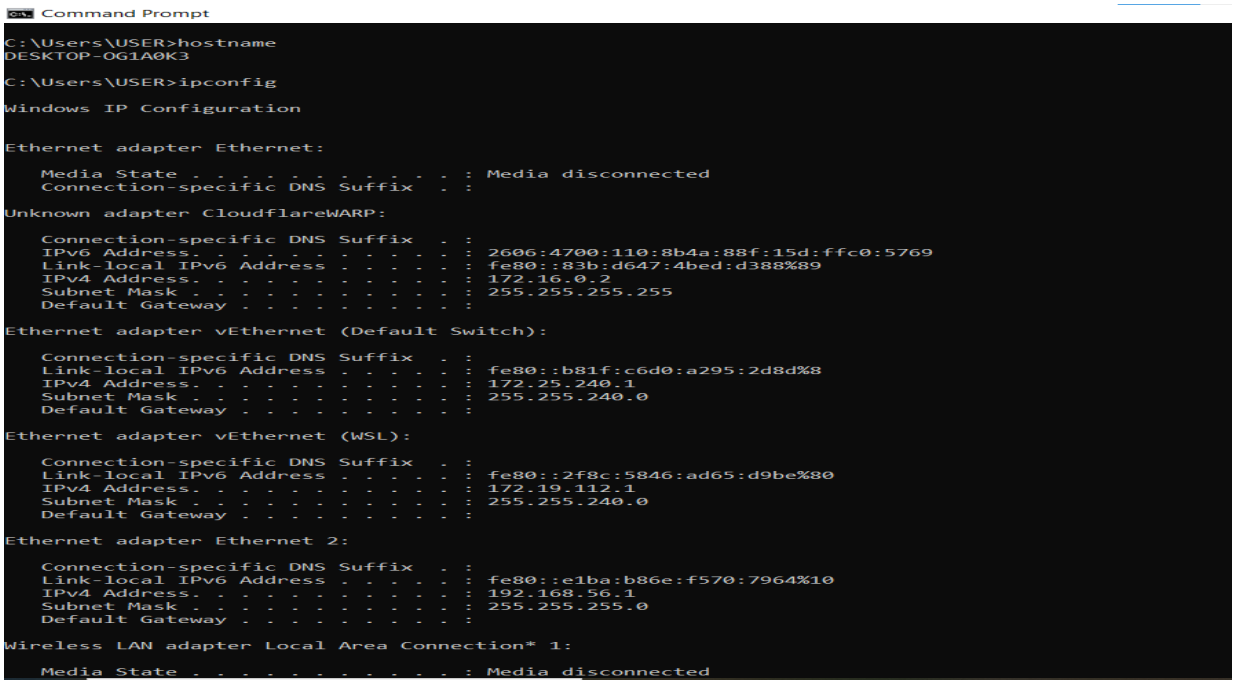
The notation 2>&1 redirects file descriptor 2 (stderr) to wherever file descriptor 1 (stdout) is currently pointing. In this task, both the successful "ls evidence" output and the error from "cat evidence/file_that_does_not_exist.txt" were captured in a single file, logs/combined_output.txt. The file contains both the directory listing ("evidence_note.txt") and the error message ("cat: evidence/file_that_does_not_exist.txt: No such file or directory"). This technique is commonly used in forensic scripting to create comprehensive audit logs that capture all command output, both successful and erroneous, in a single evidence file for streamlined documentation.

Screenshot 4: Combined stdout and stderr redirected to combined_output.txt

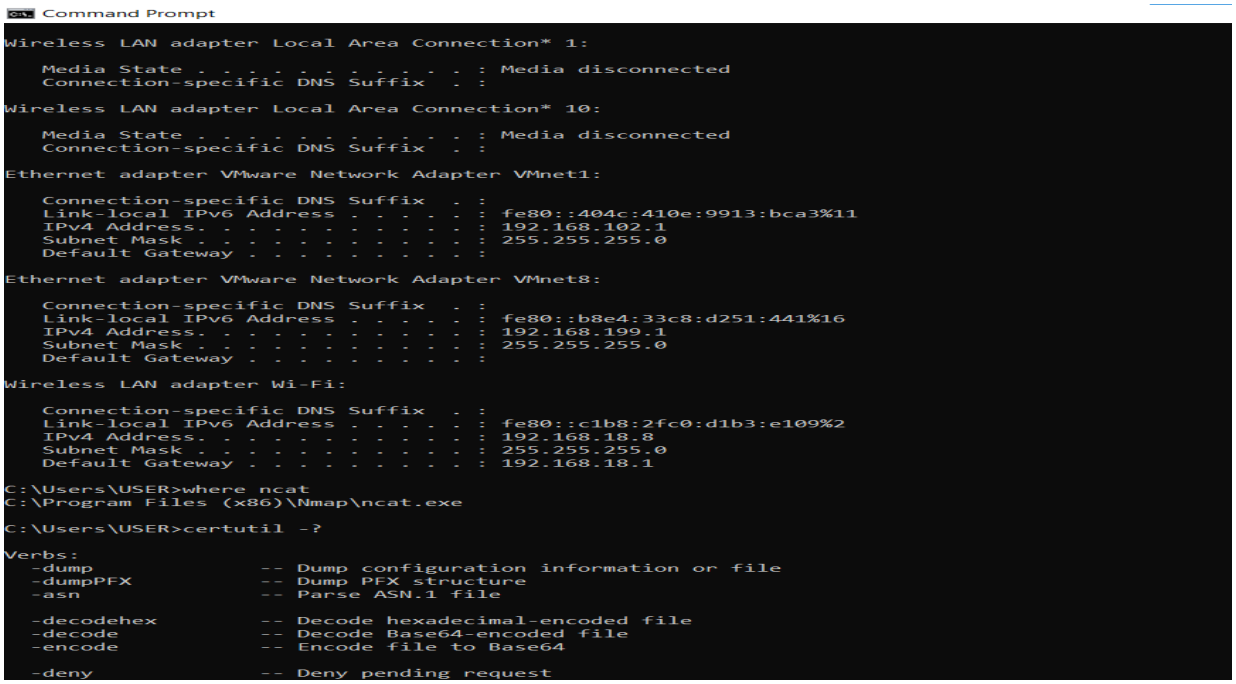
File Descriptor	Name	Abbreviation	Number	Redirection Operator
Standard Input	Keyboard/input stream	stdin	0	<
Standard Output	Normal command output	stdout	1	> or >>
Standard Error	Error/diagnostic messages	stderr	2	2> or 2>>
Combined (stderr to stdout)	All output in one file	stdout+stderr	2>&1	> file 2>&1

Part B: Baseline System and Network Documentation

Before transferring any evidence, both lab machines were documented to establish the baseline environment. This is analogous to recording acquisition environment details in a forensic notebook. The Linux machine hostname is "kali" with IP address 192.168.199.128 on the eth0 interface via a VMware NAT network. The default gateway is 192.168.199.2. Netcat is available at /usr/bin/nc. On the Windows side, the hostname is DESKTOP-OG1A0K3, and the VMware VMnet8 adapter has IP address 192.168.199.1, placing both machines on the same 192.168.199.0/24 subnet. Ncat was installed via Nmap at C:/Program Files (x86)/Nmap/ncat.exe, and certutil is available for hash computation.



Screenshot 6.1: Windows hostname and IP configuration (part 1)

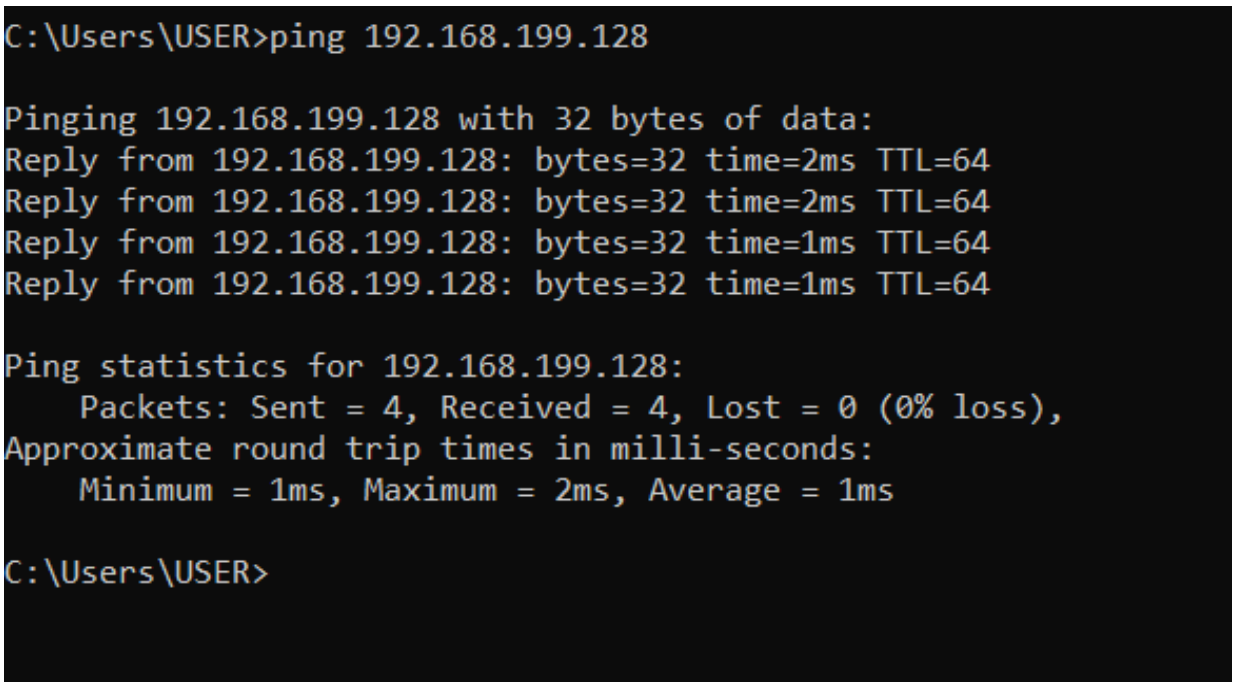


Screenshot 6.2: Windows VMnet8 IP (192.168.199.1), ncat confirmation, and certutil

Screenshot 6.3: certutil usage confirmation

B.3 Connectivity Test

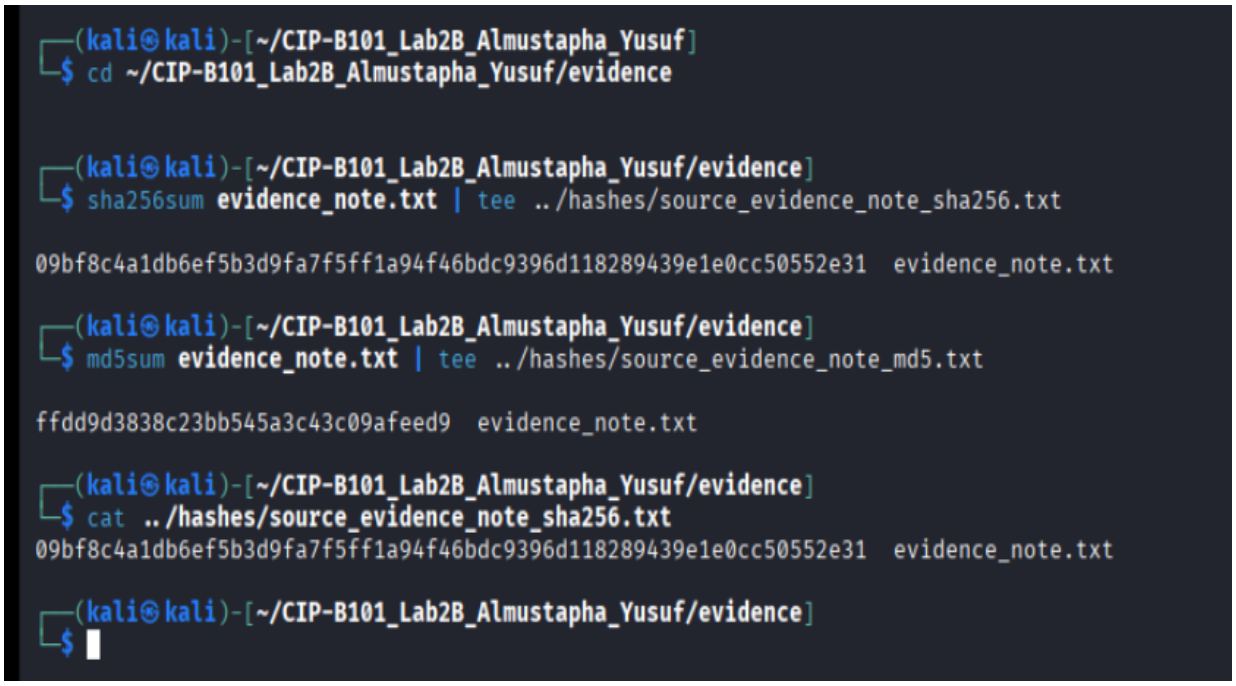
A ping test from Windows to Kali (192.168.199.128) confirmed network connectivity with 0% packet loss and an average round-trip time of 1ms. This low latency is expected since both machines communicate through the VMware virtual NAT switch without traversing physical network hardware. Successful connectivity at the network level is a prerequisite for the netcat-based evidence transfers in subsequent parts of this lab. The ping test serves as documentation that the network path between the two machines was functional at the time of the exercise.



Screenshot 7: Successful ping test from Windows to Kali (0% loss, 1ms avg)

Part C: Hash the Evidence Before Transfer

A cryptographic hash function produces a fixed-length digest that uniquely identifies the content of a file. If even a single bit of the file changes, the hash value changes completely (avalanche effect). By recording the hash before and after transfer, a forensic examiner can mathematically prove that the transferred file is identical to the source file. Two hash algorithms were used: SHA-256 (a 256-bit hash recommended by NIST for forensic applications) and MD5 (a 128-bit hash, still commonly used for quick verification despite known collision vulnerabilities). The source hashes serve as the baseline for post-transfer comparison.



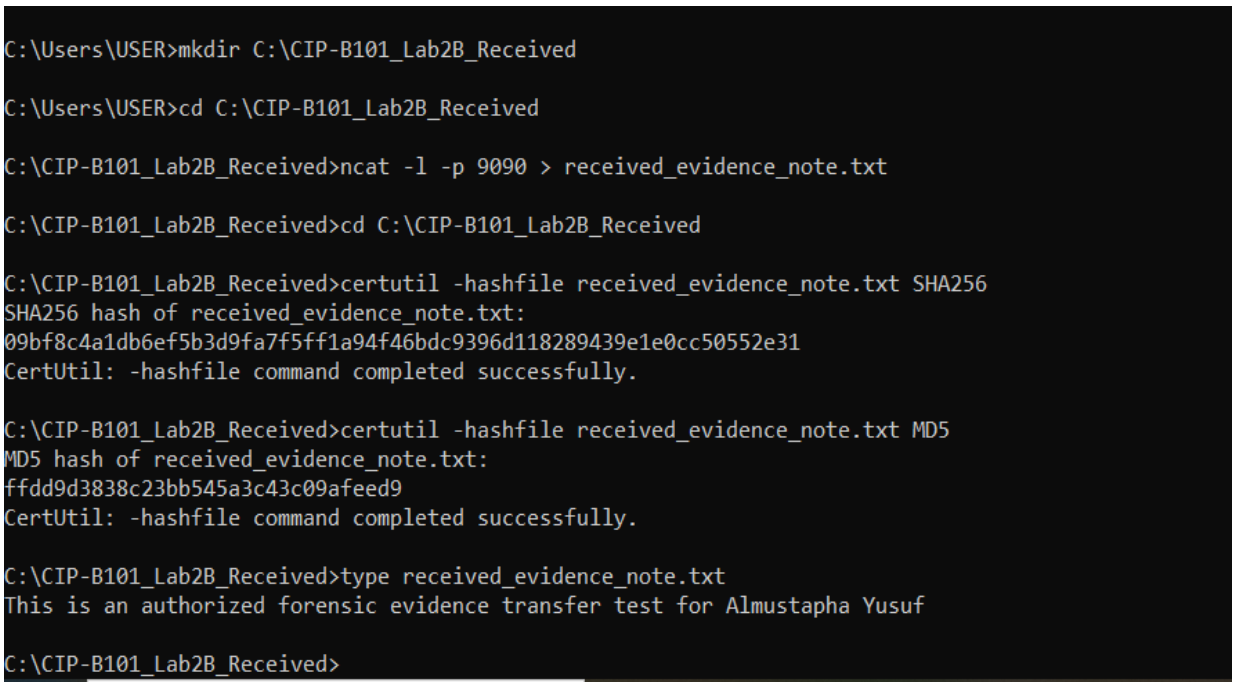
Screenshot 8: SHA256 and MD5 hashes of the source evidence file

Hash Algorithm	Source Hash Value (Linux)	File Size
SHA-256	09bf8c4a1db6ef5b3d9fa7f5ff1a94f4 6bdc9396d118289439e1e0cc50552e31	75 bytes

Hash Algorithm	Source Hash Value (Linux)	File Size
MD5	ffdd9d3838c23bb545a3c43c09afeed9	75 bytes

Part D: Controlled Evidence Transfer - Linux to Windows

This part demonstrates controlled evidence file transfer using netcat (nc) on Linux and ncat on Windows. The transfer used a simple client-server model: Windows ran ncat in listen mode on port 9090, redirecting received data to a file, while Linux connected as a client and sent the evidence file through the network connection. No shell execution options (-e or --exec) were used, ensuring this was purely a file transfer operation. The Windows listener was started first (ncat -l -p 9090 > received_evidence_note.txt), then Linux sent the file (nc 192.168.199.1 9090 < evidence_note.txt). After transfer, the received file was verified using certutil on Windows.



Screenshot 9: Windows listener, received file verification, and hash comparison

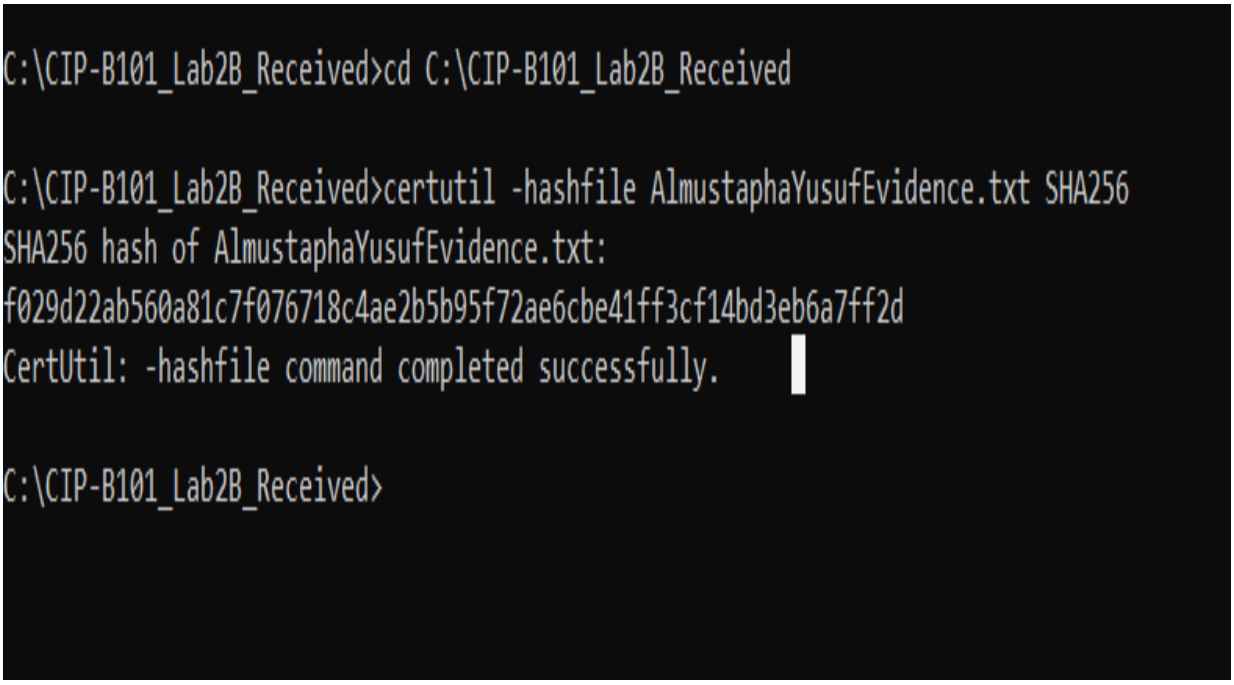
Verification	SHA-256 Hash Value	Result
Linux Source	09bf8c4a...e1e0cc50552e31	Baseline
Windows Received	09bf8c4a...e1e0cc50552e31	MATCH

Evidence Integrity Result: PASSED - The SHA-256 and MD5 hashes of the received file on Windows exactly match the source hashes computed on Linux. This confirms that the netcat transfer preserved the file content bit-for-bit with zero corruption. The evidence file is forensically intact and the transfer method is reliable.

Part E: Controlled Evidence Transfer - Windows to Linux

This part reverses the transfer direction: a file was created on Windows and sent to Linux. On Windows, the file AlmustaphaYusufEvidence.txt was created containing "hello Almustapha Yusuf!" with SHA-256 hash f029d22ab560a81c7f076718c4ae2b5b95f72ae6cbe41ff3cf14bd3eb6a7ff2d. Linux started a listener on port 9091 (nc -l -p 9091 > AlmustaphaYusufEvidence_received.txt), and Windows sent the file using ncat. After transfer, the received file on Linux was verified using sha256sum. This bidirectional transfer demonstrates that the netcat-based evidence transfer method works reliably in both directions across the lab network.

Screenshot 10: Linux received file content and SHA-256 hash verification



Screenshot 14: Windows source file SHA-256 hash for comparison

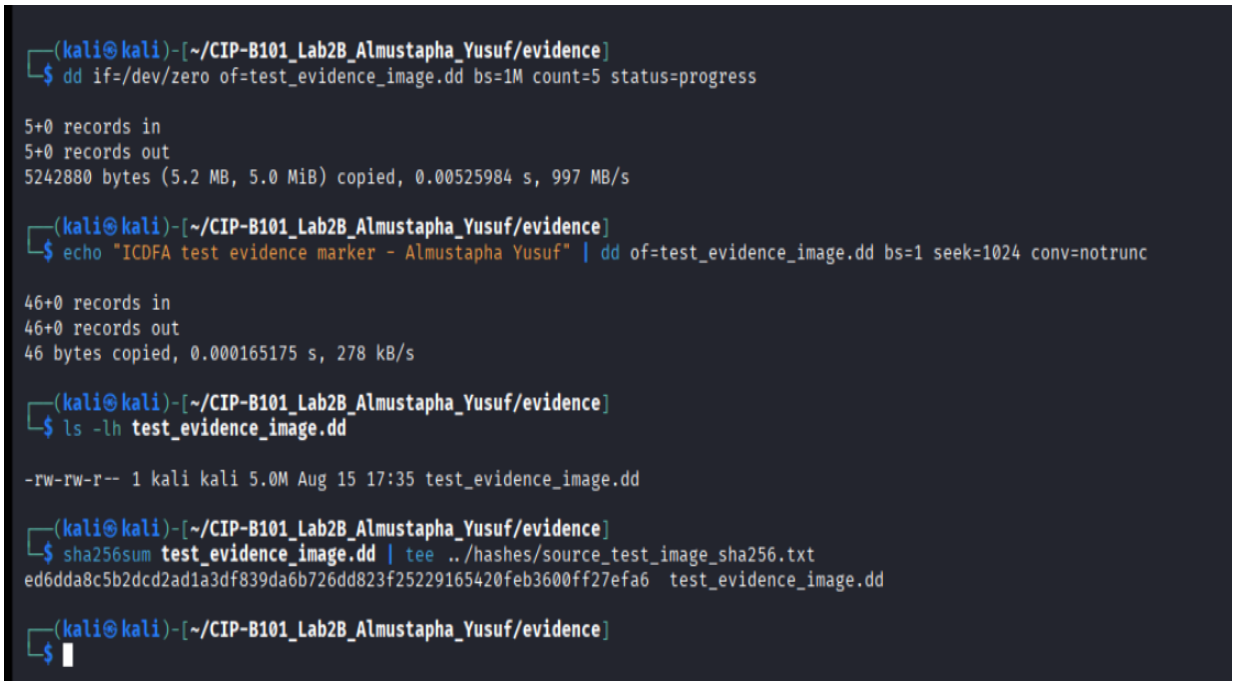
Verification	SHA-256 Hash Value	Result
Windows Source	f029d22a...f14bd3eb6a7ff2d	Baseline
Linux Received	f029d22a...f14bd3eb6a7ff2d	MATCH

Evidence Integrity Result: PASSED - The Windows-to-Linux transfer also produced a perfect hash match, confirming bidirectional forensic reliability of the netcat transfer method across the two operating systems.

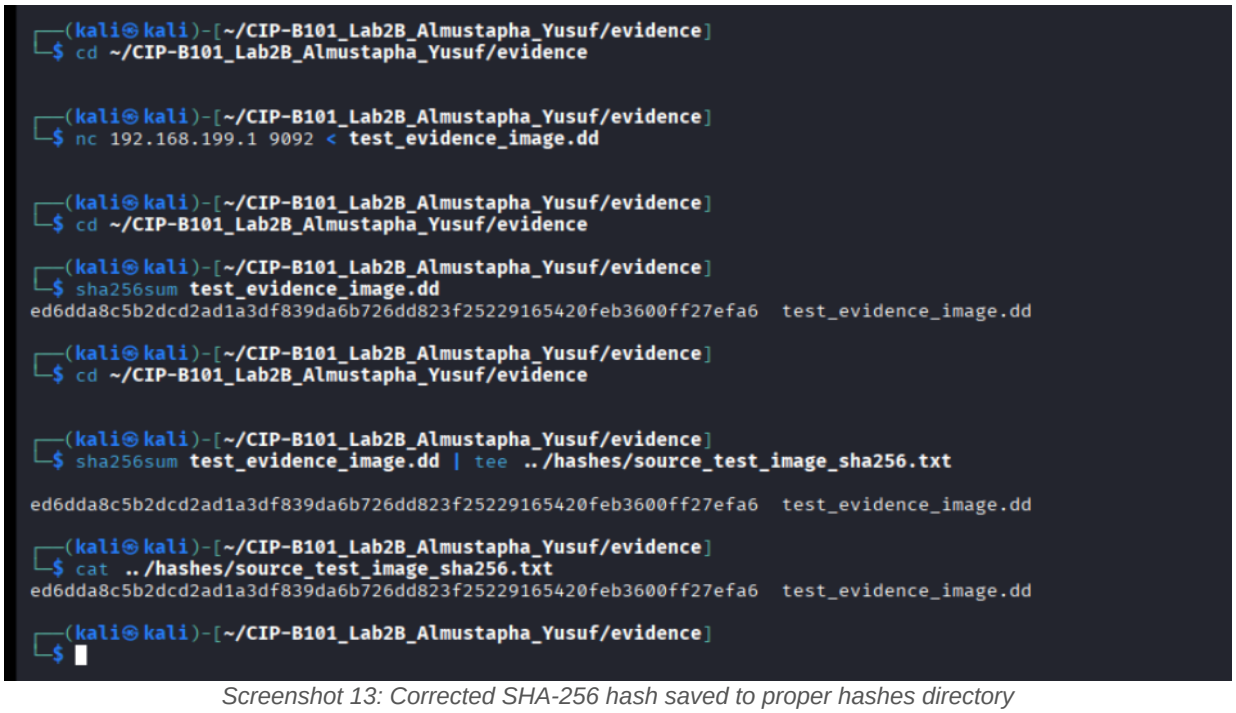
Part F: Small Test Evidence Image Transfer Using dd

The dd command is a fundamental tool for disk imaging in digital forensics. In this exercise, a small 5 MB test evidence image was created using "dd if=/dev/zero of=test_evidence_image.dd bs=1M count=5

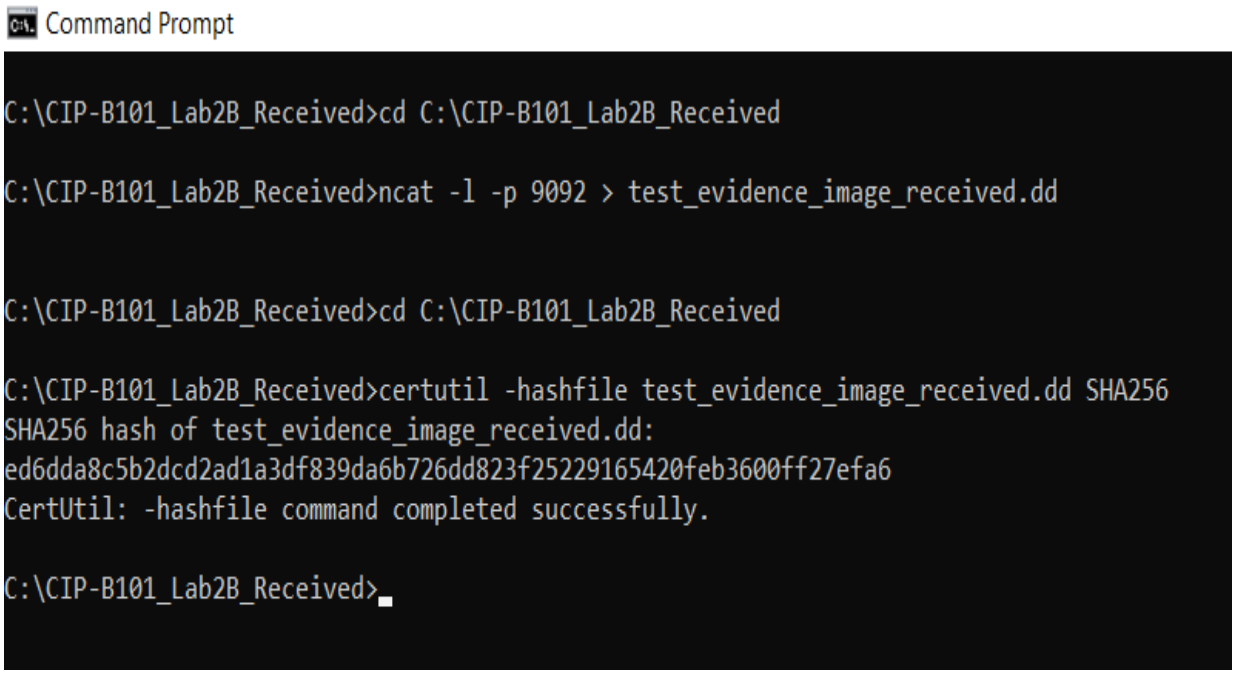
status=progress", which writes 5 megabytes of null bytes to a file. A text marker was then embedded at byte offset 1024 using "echo | dd of=test_evidence_image.dd bs=1 seek=1024 conv=notrunc". This simulates the concept of disk imaging where a forensic image contains both allocated data (the null bytes representing disk content) and specific data markers. The image was transferred from Linux to Windows via ncat on port 9092, and the SHA-256 hash was verified on both ends.



Screenshot 11: dd test image creation (5.0 MB) and source SHA-256 hash



Screenshot 13: Corrected SHA-256 hash saved to proper hashes directory



Screenshot 12: Windows verification of received dd image SHA-256 hash

Verification	SHA-256 Hash Value	Result
Linux Source (5.0 MB)	ed6dda8c5b2dcd2ad1a3df839da6b726d d823f25229165420feb3600ff27efa6	Baseline
Windows Received	ed6dda8c5b2dcd2ad1a3df839da6b726d d823f25229165420feb3600ff27efa6	MATCH

Evidence Integrity Result: PASSED - The 5 MB dd image transferred via netcat arrived on Windows with an identical SHA-256 hash, confirming that netcat-based transfer is reliable for larger binary evidence files such as forensic disk images. This validates the method for potential use with real forensic disk images.

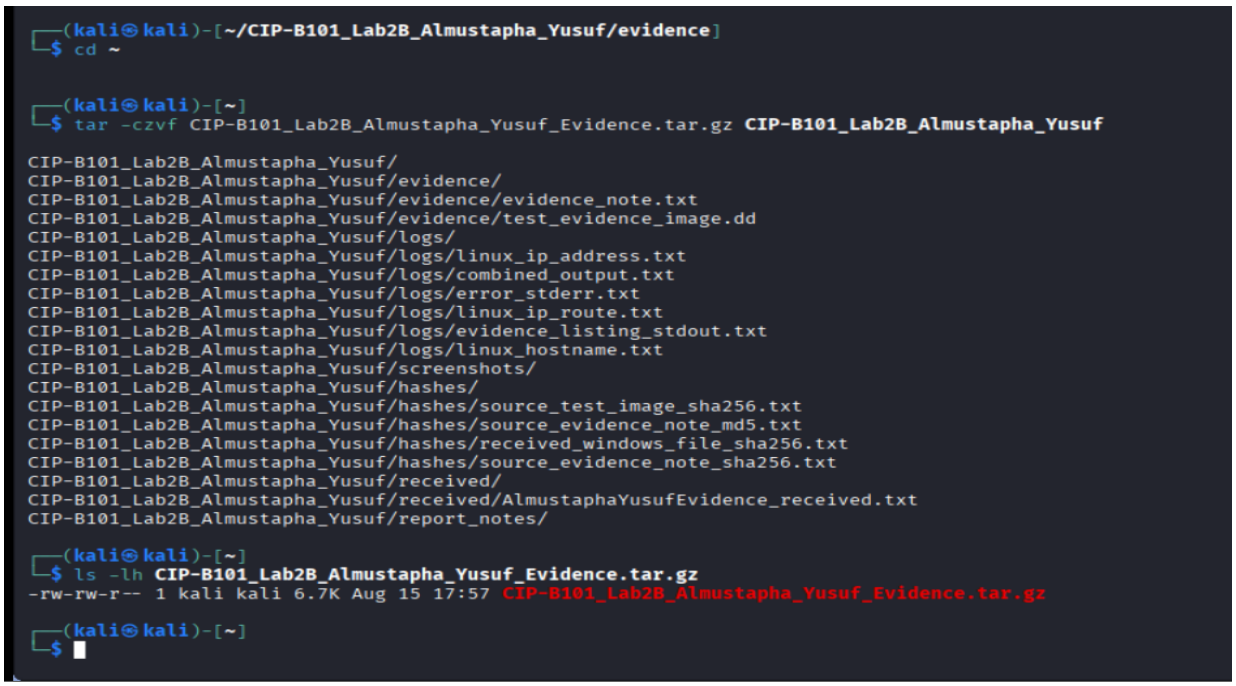
Comprehensive Evidence Integrity Summary

The following table provides a complete overview of all evidence transfers performed during this laboratory exercise, including the direction, file type, size, hash algorithm, and integrity verification result for each transfer. All three transfers achieved perfect hash matches, demonstrating the forensic reliability of netcat-based evidence transfer across a controlled lab network between Kali Linux and Windows 10.

Transfer	Direction	File	Size	Hash Algo	Result
Part D	Linux > Windows	evidence_note.txt	75 B	SHA-256 + MD5	MATCH
Part E	Windows > Linux	AlmustaphaYusufEvidence.txt	28 B	SHA-256	MATCH
Part F	Linux > Windows	test_evidence_image.dd	5.0 MB	SHA-256	MATCH

Evidence Packaging

After completing all evidence transfers and hash verifications, the entire evidence folder structure was compressed into a tar.gz archive for submission. The archive CIP-B101_Lab2B_Almustapha_Yusuf_Evidence.tar.gz contains all log files, hash files, evidence files, and received files. The compressed archive size is 6.7 KB, which is consistent with the small test files used in this exercise. In a real forensic scenario, evidence archives can be many gigabytes in size, but the packaging and integrity verification principles remain identical.



Screenshot 15: Evidence folder compressed into tar.gz archive (6.7 KB)

Reflection

1. What is the difference between standard output and standard error?

Standard output (stdout, file descriptor 1) carries the normal results of a command, such as file listings, file contents, or program output. Standard error (stderr, file descriptor 2) carries diagnostic messages, warnings, and error information. The key difference is that they use separate file descriptors, allowing them to be redirected independently. For example, "ls -la > listing.txt" captures only the directory listing in the file while errors (such as permission denied) still appear on screen. Using "2>" to redirect stderr separately allows forensic investigators to document error conditions without mixing them with legitimate command output, which is essential for maintaining clean, auditable evidence logs.

2. Why is command redirection useful during digital forensic documentation?

Command redirection is essential for forensic documentation because it creates persistent, tamper-evident records of every command executed during an investigation. Rather than relying on screenshots alone (which can be questioned in court), redirected output files serve as machine-generated evidence of exactly what commands were run and what results they produced. Redirection also enables automation through batch scripts, ensures consistency across multiple examinations, and allows forensic results to be archived, timestamped, and hashed for long-term preservation. The ability to separate stdout and stderr further enhances documentation quality by keeping normal results and error conditions in distinct, organized log files.

3. Why must a forensic examiner calculate the hash of a file before and after transfer?

Hashing before and after transfer provides mathematical proof that the file content remained unchanged during the transfer process. If the hashes match, the examiner can definitively state that no data was corrupted, modified, or lost during transit. This is critical for maintaining the chain of custody and ensuring evidence admissibility in legal proceedings. Any discrepancy between the source and destination hashes would indicate that the transfer was unreliable and the evidence would need to be re-acquired. Without hash verification, there would be no way to prove that the received file is identical to the original, undermining the entire forensic value of the evidence.

4. Why is it unsafe and unethical to use remote shell access on a system without authorization?

Using remote shell access without authorization is illegal in most jurisdictions under computer crime and unauthorized access laws. It violates the principle of consent and can cause harm to systems and data. From a forensic perspective, unauthorized access contaminates the evidence because the examiner cannot distinguish between actions performed by the legitimate user and actions performed by the unauthorized access tool. This violates chain of custody principles and can render evidence inadmissible. Ethically, forensic examiners must operate within the bounds of their authorization to maintain professional integrity, public trust, and legal compliance. Any forensic technique must be justified, documented, and performed only on systems for which explicit authorization has been granted.

5. What is the difference between a bind shell and a reverse shell at a conceptual level?

A bind shell opens a listening port on the target machine and waits for an attacker to connect to it. The target machine "binds" to a port and serves a shell to whoever connects. A reverse shell is the opposite: the target machine initiates an outbound connection to the attacker machine and presents a shell. Reverse shells are often used to bypass firewalls because outbound connections from the target are more likely to be permitted than inbound connections. From a forensic perspective, detecting a bind shell involves looking for unexpected listening ports (using netstat or ss), while detecting a reverse shell involves analyzing outbound network connections and identifying processes that should not be initiating external connections. Both are serious security indicators that forensic investigators must be able to recognize and analyze.

6. Why might a firewall block one type of connection but allow another?

Firewalls typically enforce rules based on connection direction, port numbers, and protocol types. Most firewalls are configured to block unsolicited inbound connections while allowing outbound connections initiated from inside the network. This is why reverse shells (outbound from target) often succeed where bind shells (inbound to target) are blocked. Additionally, firewalls may allow traffic on common ports (80, 443) while blocking less common ports. In a forensic investigation, understanding firewall rules helps explain why certain network connections succeeded or failed, and can indicate whether an attacker used specific techniques to evade firewall restrictions. This knowledge also helps forensic investigators recommend appropriate firewall configurations to prevent future unauthorized access.

7. Why did this lab use a small test evidence image instead of imaging a real disk?

Using a small 5 MB test image instead of a real disk image ensures student safety, avoids potential damage to the host system, keeps transfer times manageable for a lab exercise, and eliminates legal and ethical concerns associated with imaging real storage devices. The principles demonstrated (dd creation, hash verification, netcat transfer, integrity comparison) are identical regardless of image size. A real disk could contain sensitive personal data, and imaging it without proper authorization would violate privacy regulations. The test image teaches the same forensic acquisition workflow while keeping the exercise safe, legal, and focused on the learning objectives. In a professional setting, these same techniques would be applied to full disk images with proper write-blocking and authorization.